

Module: epd4spi

Functions:

Function: init

```
void Epd4Spi::init(uint8_t frequency=4, bool debug=false)
```

Inicializa a comunicação SPI com configurações específicas.

Details: Olá, Time LEGACY DOC! Neste método, eu configuro os pinos do SPI e defino suas direções. Isso garante que a comunicação SPI funcione corretamente, permitindo que os dispositivos se comuniquem de forma eficaz. Além disso, habilito o modo de depuração, se necessário, para facilitar a resolução de problemas.

Parameters:

- frequency (uint8_t): Frequência do SPI.
- debug (bool): Ativa o modo de depuração.

Function: gpio_set_direction

```
void gpio_set_direction(gpio_num_t gpio_num, gpio_mode_t mode)
```

Configura a direção de um pino GPIO.

Details: Olá, Time LEGACY DOC! Este método é utilizado para definir se um pino GPIO será uma entrada ou saída. Isso é crucial para garantir que os pinos funcionem corretamente em um sistema, permitindo a comunicação adequada entre os componentes eletrônicos. A configuração correta dos pinos é fundamental para o funcionamento do dispositivo.

Parameters:

- gpio_num (gpio_num_t): Número do GPIO a ser configurado.
- mode (gpio_mode_t): Modo do GPIO (entrada ou saída).

Function: gpio_set_pull_mode

```
void gpio_set_pull_mode(gpio_num_t gpio_num, gpio_pull_mode_t pull_mode)
```

Configura o modo de pull de um pino GPIO.

Details: Olá, Time LEGACY DOC! Aqui, eu configuro o modo de pull de um pino GPIO, que é essencial para garantir que o pino tenha um estado definido quando não está sendo acionado. Isso ajuda a evitar flutuações indesejadas e garante uma comunicação estável entre os componentes do sistema. A configuração correta é

vital para o desempenho do dispositivo.

Parameters:

- gpio_num (gpio_num_t): Número do GPIO a ser configurado.
- pull_mode (gpio_pull_mode_t): Modo de pull do GPIO.

Function: cmdM1

```
void Epd4Spi::cmdM1(const uint8_t cmd)
```

Envia um comando para o dispositivo M1.

Details: Olá, Time LEGACY DOC! Neste método, eu envio um comando para o dispositivo M1. Primeiro, verifico se o modo de depuração está ativado e, se sim, imprimo o comando. Em seguida, configuro a transação SPI e transmito o comando. Por fim, restauro os níveis dos pinos de controle.

Parameters:

- cmd (uint8_t): Comando a ser enviado para o dispositivo.

Function: dataM1

```
void Epd4Spi::dataM1(uint8_t data)
```

Envia dados para o dispositivo M1.

Details: Olá, Time LEGACY DOC! Neste método, eu envio dados para o dispositivo M1. Assim como no método anterior, verifico o modo de depuração e imprimo os dados. Depois, configuro a transação SPI e transmito os dados. Finalmente, restauro os níveis dos pinos de controle.

Parameters:

- data (uint8_t): Dados a serem enviados para o dispositivo.

Function: cmdS1

```
void Epd4Spi::cmdS1(const uint8_t cmd)
```

Envia um comando S1 via SPI.

Details: Olá, Time LEGACY DOC! Neste método, eu envio um comando S1 através da interface SPI. Primeiro, verifico se o modo de depuração está ativado e, em seguida, configuro a transação SPI. Após a transmissão, restauro os níveis de GPIO para finalizar o comando.

Parameters:

- cmd (uint8_t): O comando a ser enviado via SPI.

Function: dataS1

```
void Epd4Spi::dataS1(uint8_t data)
```

Envia dados S1 via SPI.

Details: Olá, Time LEGACY DOC! Neste método, eu envio dados S1 pela interface SPI. Primeiro, verifico se o modo de depuração está ativado e, em seguida, configuro a transação SPI. Após a transmissão, restauro os níveis de GPIO para concluir o envio dos dados.

Parameters:

- data (uint8_t): Os dados a serem enviados via SPI.

Function: cmdM2

```
void Epd4Spi::cmdM2(const uint8_t cmd)
```

Envia um comando M2 via SPI.

Details: Olá, Time LEGACY DOC! Neste método, eu envio um comando M2 através da interface SPI. Primeiro, verifico se o modo de depuração está ativado e, em seguida, configuro a transação SPI. O método finaliza a transmissão do comando, garantindo que a comunicação SPI ocorra corretamente.

Parameters:

- cmd (uint8_t): O comando a ser enviado via SPI.

Function: dataS1

```
void Epd4Spi::dataS1(uint8_t data)
```

Envia dados S1 via SPI.

Details: Olá, Time LEGACY DOC! Neste método, eu envio dados S1 pela interface SPI. Primeiro, verifico se o modo de depuração está ativado e, em seguida, configuro a transação SPI. Após a transmissão, restauro os níveis de GPIO para concluir o envio dos dados.

Parameters:

- data (uint8_t): Os dados a serem enviados via SPI.

Function: dataM2

```
void Epd4Spi::dataM2(uint8_t data)
```

Envia dados via SPI.

Details: Olá, Time LEGACY DOC! Neste método, eu envio um dado via SPI. Primeiro, verifico se o modo de

depuração está ativado e imprimo o dado. Em seguida, configuro os níveis GPIO, preparo a transação SPI e transmito o dado. Por fim, restauro o nível GPIO.

Parameters:

- data (uint8_t): O dado a ser enviado via SPI.

Function: cmdS2

```
void Epd4Spi::cmdS2(const uint8_t cmd)
```

Envia um comando via SPI.

Details: Olá, Time LEGACY DOC! Neste método, eu envio um comando via SPI. Primeiro, verifico se o modo de depuração está ativado e imprimo o comando. Depois, configuro os níveis GPIO, preparo a transação SPI e transmito o comando. Por fim, restauro o nível GPIO.

Parameters:

- cmd (const uint8_t): O comando a ser enviado via SPI.

Function: dataM2

```
void Epd4Spi::dataM2(uint8_t data)
```

Envia dados via SPI.

Details: Olá, Time LEGACY DOC! Neste método, eu envio um dado via SPI. Primeiro, verifico se o modo de depuração está ativado e imprimo o dado. Em seguida, configuro os níveis GPIO, preparo a transação SPI e transmito o dado. Por fim, restauro o nível GPIO.

Parameters:

- data (uint8_t): O dado a ser enviado via SPI.

Function: cmdS2

```
void Epd4Spi::cmdS2(const uint8_t cmd)
```

Envia um comando via SPI.

Details: Olá, Time LEGACY DOC! Neste método, eu envio um comando via SPI. Primeiro, verifico se o modo de depuração está ativado e imprimo o comando. Depois, configuro os níveis GPIO, preparo a transação SPI e transmito o comando. Por fim, restauro o nível GPIO.

Parameters:

- cmd (const uint8_t): O comando a ser enviado via SPI.

Function: dataS2

```
void Epd4Spi::dataS2(uint8_t data)
```

Envia dados para o display S2.

Details: Olá, Time LEGACY DOC! Neste método, eu envio dados para o display S2. Primeiro, habilito o modo de depuração, se necessário. Em seguida, configuro a transação SPI e transmito os dados. Isso garante que o display receba as informações corretas para funcionar corretamente.

Parameters:

- data (uint8_t): Dados a serem enviados para o display.

Function: cmdM1S1M2S2

```
void Epd4Spi::cmdM1S1M2S2(uint8_t cmd)
```

Envia um comando para todos os 4 displays.

Details: Olá, Time LEGACY DOC! Aqui, eu envio um comando para todos os quatro displays. Após configurar os pinos de controle, transmito o comando via SPI. Isso assegura que todos os displays respondam ao mesmo comando simultaneamente, facilitando a sincronização.

Parameters:

- cmd (uint8_t): Comando a ser enviado para todos os displays.

Function: dataM1S1M2S2

```
void Epd4Spi::dataM1S1M2S2(uint8_t data)
```

Envia dados para os displays M1, S1, M2 e S2.

Details: Olá, Time LEGACY DOC! Neste método, eu envio dados para os displays M1, S1, M2 e S2. Embora o modo de depuração esteja comentado, a configuração da transação SPI e a transmissão dos dados são realizadas para garantir que todos os displays recebam as informações necessárias.

Parameters:

- data (uint8_t): Dados a serem enviados para os displays.

Function: data

```
void Epd4Spi::data(const uint8_t *data, int len)
```

Envia dados para o SPI.

Details: Olá, Time LEGACY DOC! Neste método, eu envio dados para o SPI chamando a função dataM1.

Legacy Doc - Documentação Técnica

Isso garante que os dados sejam transmitidos corretamente, utilizando a configuração adequada do SPI. O impacto é que a comunicação com o dispositivo SPI se torna eficiente e confiável.

Parameters:

- data (const uint8_t *): Um ponteiro para os dados a serem enviados.
- len (int): O comprimento dos dados em bytes.

Function: dataM1

```
void Epd4Spi::dataM1(const uint8_t *data, int len)
```

Transmite dados via SPI usando polling.

Details: Olá, Time LEGACY DOC! Neste método, eu transmito dados via SPI, garantindo que a transação seja feita corretamente. Ao utilizar o modo polling, eu consigo uma transmissão mais rápida, evitando a sobrecarga de interrupções. Isso resulta em uma comunicação mais eficiente com o dispositivo.

Parameters:

- data (const uint8_t *): Um ponteiro para os dados a serem enviados.
- len (int): O comprimento dos dados em bytes.

Function: dataS1

```
void Epd4Spi::dataS1(const uint8_t *data, int len)
```

Transmite dados usando o canal S1.

Details: Olá, Time LEGACY DOC! Neste método, verifico se o comprimento é zero. Se não for, configuro o nível do GPIO para iniciar a transmissão, transmito os dados e, em seguida, restauro o nível do GPIO. Isso garante que os dados sejam enviados corretamente pelo canal S1.

Parameters:

- data (const uint8_t *): Um ponteiro para os dados a serem transmitidos.
- len (int): O comprimento dos dados a serem transmitidos.

Function: dataS2

```
void Epd4Spi::dataS2(const uint8_t *data, int len)
```

Transmite dados usando o canal S2.

Details: Olá, Time LEGACY DOC! Aqui, verifico se o comprimento é zero. Se não for, configuro o nível do

GPIO para iniciar a transmissão, transmito os dados e restauro o nível do GPIO. Isso assegura que os dados sejam enviados corretamente pelo canal S2.

Parameters:

- data (const uint8_t *): Um ponteiro para os dados a serem transmitidos.
- len (int): O comprimento dos dados a serem transmitidos.

Function: reset

```
void Epd4Spi::reset(uint8_t millis=20)
```

Reseta o dispositivo com um atraso especificado.

Details: Olá, Time LEGACY DOC! Neste método, configuro os níveis do GPIO para zero, aguardo um tempo especificado e, em seguida, restauro os níveis do GPIO. Isso garante que o dispositivo seja resetado corretamente.

Parameters:

- millis (uint8_t): O tempo em milissegundos para o reset.